Distributed Machine Learning for Fake News Detection Using Apache Spark MLlib

Savannah G. Diggett

Motivation

Fake news detection is the process of identifying misinformation to verify the accuracy of news content. Minimizing the spread of misinformation is critical as it poses a direct threat to democratic integrity and public health. Manual fact-checking is increasingly infeasible at scale, motivating the need for automated machine learning solutions.

This is inherently a classification problem: a model is trained on labelled historical data and used to predict whether unseen statements are real or fake. Text data is unstructured and cannot be fed directly into a model, requiring feature engineering techniques such as TF-IDF to convert raw text into numerical representations. Simple rule-based approaches are insufficient because fake news often mimics the language of real news, making the distinction subtle.

This problem is particularly relevant in a big data context as social media platforms generate millions of statements daily, requiring a scalable distributed solution. Spark MLlib is well suited to this task as it can scale horizontally across a cluster of machines, making it applicable to real-world content moderation pipelines.

Data Processing & Feature Engineering

Dataset Description

The LIAR dataset (4) contains 12,836 short political statements labelled across 6 veracity classes: true, mostly-true, half-true, barely-true, false, and pants-fire. The categorical variables were converted to integers.

Data Cleaning

As the data did not have headers, the schema was defined manually, and missing values were replaced with "unknown." String values were converted to numeric with Spark's `StringIndexer` function.

Feature Engineering

As the data initially had 6 classes (e.g. mostly-true, half-true, etc.), it would be much harder for the classification estimator to identify distinct boundaries between such similar categories, so the 6 classes were converted into a binary label (real (0) or fake (1)) by grouping true, mostly-true, and half-true as real, and barely-true, false, and pants-fire as fake. This simplifies the classification task, produces a more balanced class distribution, and reflects the practical goal of fake news detection, which is ultimately a binary decision.

To convert the unstructured text data into a structured numerical format suitable for machine learning, a series of Spark ML transformers was applied. First, `Tokenizer` splits each raw statement into individual words, after which `StopWordsRemover` eliminates common words such as "and," "the," and "him" to ensure only meaningful terms are retained. `HashingTF` then converts the remaining words into a term frequency vector of 10,000 features, and `IDF`

downweights terms that appear frequently across all documents, upweighting rarer and more informative terms.

For categorical variables such as speaker and party, `FeatureHasher` (5) was applied to map these high-cardinality features into a fixed-size numerical vector of 1,000 features. This was chosen over `OneHotEncoder` to avoid the curse of dimensionality, as the large number of unique speakers and parties would otherwise create an unmanageable number of columns. For example, if there were 5,000 unique speakers in the dataset, `OneHotEncoder` would create 5,000 additional binary columns (one for each speaker), causing the feature space to explode in size, which increases computational cost and can actually reduce model performance as the data becomes increasingly sparse across such a large number of dimensions. Finally, `VectorAssembler` combines all text, categorical, and numerical features into a single feature vector as input to the model.

Exploratory Analysis

Prior to model training, the correlation between the numerical credibility count features and the binary label was examined to assess their predictive value. The results revealed very weak correlations across all five features, with `pants_fire` showing the strongest relationship at 0.16, followed by `mostly_true` at -0.08 and `half_true` at -0.07. The remaining features, `barely_true` and `false_count`, showed correlations of 0.01 and 0.06 respectively.

These near-zero correlations suggest that a speaker's historical credibility counts alone are insufficient to reliably predict whether any individual statement is fake. This is likely because the counts reflect a speaker's overall credibility rather than the content of a specific statement — a generally honest speaker can still make a false claim, and vice versa.

This finding reinforces the importance of text features in this classification task. Since the statement content itself carries the most direct signal about veracity, the TF-IDF text features derived from the raw statement text are essential to model performance. Without them, the model would be relying almost entirely on speaker metadata, which as shown above has very limited predictive power on its own.

Baseline Model (Logistic Regression)

The Logistic Regression classifier was chosen because it is simple, interpretable, and has been proven to be an adequate baseline estimator for text data (2). The data met the assumptions for a logistic regression, which include: linearity, independence of observations, and no multi-collinearity. The training and testing split was already supplied and the test set made up 10% of the data, which is an appropriate split (3). The Spark ML Logistic Regression pipeline is as follows:

Stage 1: `Tokenizer` (Transformer) → splits statement text into words

Stage 2: `StopWordsRemover` (Transformer) → removes common uninformative words

Stage 3: `HashingTF` (Transformer) → converts words to term frequency vector (10,000 features)

Stage 4: `IDF` (Estimator) → downweights common terms across documents

Stage 5: `FeatureHasher` (Transformer) → hashes speaker + party into vector (1,000 features)

Stage 6: `VectorAssembler` (Transformer) → combines all features into a single vector

Stage 7: `LogisticRegression` (Estimator) → trains the final classification model

The output variable is called `binary_label` with 0 = real and 1 = fake. The Logistic Regression model did slightly better than chance at accurately identifying fake news. The AUC was 56.33% and accuracy was 55.11%. AUC and accuracy were selected as the primary metrics to report, as AUC handles imbalance better and accuracy scores are easier to interpret.

Advanced Model (Gradient Boosted Trees)

Three different advanced models/ensemble methods were employed: Random Forest (bagging algorithm), Gradient Boosted Trees (boosting), as well as a stacking algorithm that combined Logistic Regression and Random Forest, as they complement each other's weaknesses.

Model Choice

For conciseness, this report only reports the results from one of the methods, as required in the guidelines. GBT was selected over Random Forest because it is a boosting method that trains trees sequentially, with each tree correcting the errors of the previous one. This iterative error-correction mechanism is particularly well suited to this dataset because the signal for fake news detection is subtle, as no single feature is highly predictive on its own, and the relationship between features and the target is complex and non-linear. GBT's sequential error correction is good at picking up on the subtle combinations of speaker metadata and text patterns that distinguish fake from real statements. By gradually reducing bias through sequential refinement, GBT is able to capture these patterns more effectively than a single model or a bagging approach. In terms of the bias-variance tradeoff, GBT primarily addresses bias by combining many weak learners into one strong model. Unlike Random Forest, which reduces variance by averaging many independent trees, GBT focuses on iteratively improving the parts of the data the model gets wrong, making it more aggressive at learning difficult patterns. The key hyperparameters selected were `maxIter=30`, controlling the number of sequential trees, and `maxDepth=5`, which controls the complexity of each individual tree. These were chosen to balance model complexity against the risk of overfitting on the relatively small LIAR dataset.

Design Decisions

All three models were trained on the same feature pipeline — `Tokenizer`, `StopWordsRemover`, `HashingTF`, `IDF`, `FeatureHasher`, and `VectorAssembler` — ensuring a fair comparison where the only variable is the choice of classifier. The key hyperparameters for each model were selected as follows:

For Random Forest, `numTrees=100` was chosen to ensure sufficient diversity across trees without excessive computational cost, and `maxDepth=10` allows each tree to capture reasonably complex patterns while limiting overfitting.

For GBT, `maxIter=30` controls the number of sequential boosting rounds — set conservatively to balance performance against training time on the relatively small LIAR dataset — and `maxDepth=5` keeps individual trees shallow, which is standard practice in boosting to avoid overfitting since the ensemble itself provides the complexity.

Hypotheses

GBT was expected to perform the best out of all the models tested, but to be the slowest to train due to its sequential nature. It was predicted to significantly outperform Logistic Regression because, unlike LR which fits a single linear decision boundary, GBT builds trees one after another where each tree focuses on correcting the mistakes of the last one. This means

Figure 1: GBT training time vs. data size on GCP cluster.

it should be much better at picking up on the complex and non-linear relationships between the text features, speaker metadata, and whether a statement is fake or not.

Logistic Regression was expected to perform only slightly better than random guessing on this dataset, as the linearity assumption is unlikely to hold for high-dimensional sparse TF-IDF features combined with the subtle patterns that separate fake from real news.

Evaluation

In terms of evaluation metrics, this study utilized three: the area under the curve, accuracy, and F1 score. AUC was chosen as the primary metric because it measures the model's ability to distinguish between real and fake statements across all classification thresholds. Accuracy was included as an evaluation metric because it represents the proportion of correctly classified statements. Due to the balanced class distribution (56% vs. 44%), accuracy is an appropriate metric. The F1 score is important for fake news detection because both false negatives (flagging real news as fake) and false positives (flagging fake news as real) carry importance in the real world.

While more computationally expensive, GBT provided significantly greater improvements in terms of accuracy over the baseline Logistic Regression.

Model	**AUC**	**Accuracy**	**F1**	**Train Time**
Logistic Regression	0.56	0.55	0.55	2.58 s
GBT	0.81	0.73	0.73	53.79 s

Table 1: Model performance comparison.

GBT significantly outperforms Logistic Regression across every metric. LR's AUC of 0.56

is barely better than random guessing, which is expected — fake news detection is inherently non-linear and interaction-dependent; for example, the same words can indicate fake news when spoken by one speaker but not another, which a linear model simply cannot capture. GBT's AUC of 0.81 and accuracy of 73.3% are strong results that exceed the typical state of the art of 65–70% on the LIAR dataset, likely because combining TF-IDF text features with speaker credibility metadata provides a richer signal than text alone. The near-identical precision and recall scores (both 0.73) also suggest the model is well balanced — it is neither over-flagging real news as fake nor missing too many fake statements, which is exactly what you want from a content moderation system.

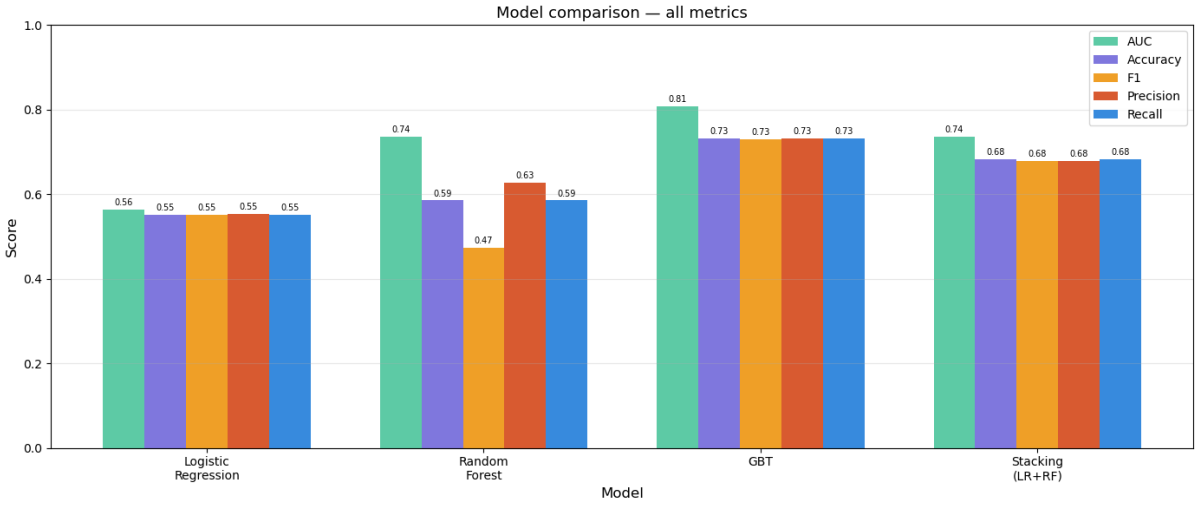

Figure 2: Comparison of model performance.

Comparative Scenario

Notably, GBT did not converge on the single-node Colab environment within a reasonable time, further demonstrating that computationally expensive sequential ensemble methods require distributed infrastructure to be practical at even modest dataset sizes. A key motivation for using Spark ML is its ability to distribute training across multiple worker nodes in a cluster. All three models benefit from this, as Spark partitions the training data across nodes so that each worker processes a subset in parallel. However, the computational cost varies significantly between models.

Environment	**Nodes**	**LR Training Time**
Google Colab	1	5.97 seconds
GCP Dataproc	3 (1 master, 2 workers)	2.58 seconds

Table 2: Training environment comparison for Logistic Regression.

Logistic Regression is the most scalable, as it converged quickly (2.58 seconds) and communicates minimal information between nodes. Random Forest is moderately expensive (13.55 seconds) to train, but highly parallelizable since each tree is independent and can be trained on a separate node simultaneously. GBT is the most computationally expensive (53.79 seconds) to train because its sequential nature means each round must complete before the next begins, limiting the degree of parallelism. This represents the fundamental scalability tradeoff, as LR is fast and highly scalable but underfits complex patterns, while GBT is slower and more resource intensive but achieves the highest predictive performance.

Training on the GCP Dataproc cluster was approximately 2.3 times faster than on a single node, demonstrating the tangible benefit of distributed computing even on a relatively small dataset. The cluster achieves this by partitioning the training data across worker nodes, allowing each node to process a subset of the data simultaneously. This speedup would be significantly more pronounced on larger datasets; at the scale of millions of social media posts, a single machine would become a bottleneck entirely, making distributed computing not just beneficial but necessary. GBT was not included in this comparison as it did not converge on a single node within a reasonable time, which itself further demonstrates the necessity of distributed infrastructure for more computationally expensive models.

Limitations

While a very high accuracy was achieved for the LIAR dataset, the solution still suffers from a few notable limitations. Firstly, the LIAR dataset is a relatively small dataset for a big pipeline, as it only contains 12k rows; this is moderately small for a distributed Spark pipeline. The scalability benefits are therefore modest at this size.

Secondly, when the six labels were collapsed into two, this introduced a lot of noise into the data as relatively ambiguous statements were forced into discrete categories. Thirdly, the statements in the LIAR dataset were relatively short, averaging around 17 words; this produces very sparse TF-IDF vectors where most of the 10k features are zero for any given statement. However, despite these limitations, the pipeline demonstrates that a scalable and effective fake news detection system can be built using Spark MLlib.

Conclusion

In this study, a scalable fake news detection pipeline was built using Spark MLlib, with GBT achieving an AUC of 0.81 and accuracy of 73.3%, significantly outperforming the Logistic Regression baseline, which scored only 0.56 AUC, while the distributed GCP cluster reduced training time by 2.3 times compared to a single-node environment. These results suggest that with access to richer data, such as full article text, source metadata, and social network propagation patterns, the approach could be meaningfully extended into a production-ready content moderation system. More broadly, this study demonstrates that Spark MLlib provides a viable and scalable foundation for NLP-based classification tasks, and that the pipeline architecture designed here could be readily adapted to other misinformation detection problems beyond political statements.

References

[1] Darbon, J., & Osher, S. (2016). Algorithms for overcoming the curse of dimensionality for certain Hamilton–Jacobi equations arising in control theory and elsewhere. *Research in the Mathematical Sciences*, 3(1), 1–26. https://doi.org/10.1186/s40687-016-0068-7

[2] Huang, J.-Y., & Liu, J.-H. (2020). Using social media mining technology to improve stock price forecast accuracy. *Journal of Forecasting*, 39(1), 104–116. https://doi.org/10.1002/for.2616

[3] Sivakumar, M., Parthasarathy, S., & Padmapriya, T. (2024). Trade-off between training and testing ratio in machine learning for medical image processing. *PeerJ Computer Science*, 10, e2245. https://doi.org/10.7717/peerj-cs.2245

[4] Wang, W. Y. (2017). "Liar, liar pants on fire": A new benchmark dataset for fake news detection. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 422–426. https://doi.org/10.18653/v1/P17-2067

[5] Weinberger, K., Dasgupta, A., Langford, J., Smola, A., & Attenberg, J. (2009). Feature hashing for large scale multitask learning. In *Proceedings of the 26th Annual International Conference on Machine Learning, ICML 2009*, Montreal, Quebec, Canada, June 14–18, 2009, pages 1113–1120.